

Derived, Not Measured: Closed-Form Memory Behavior for Affine Kernels

1. A fifty-year asymmetry

Ask what a matrix multiplication costs in arithmetic and you get a formula: $2n^3$ operations. Symbolic, exact, valid for every n , derived by inspection. Ask what the *memory system* will do — the question that actually decides performance on modern hardware, where fetching a value from main memory costs tens to hundreds of times more than multiplying it — and for fifty years the only honest answer has been: *run it and see*.

This is not for lack of theory. Since Mattson et al. (1970) we have known the right quantity: for each access, count the distinct data touched since the *previous* access to the same value — its **reuse distance**. A cache holding C values serves an access from fast memory exactly when its reuse distance is at most C ; otherwise the access goes to main memory (a *miss*). The histogram of reuse distances therefore determines the miss ratio of *every* cache size at once, and for a simple kernel you can sketch it by hand. Naive matmul touches $C[i][j]$ again after ~ 3 values (the running sum), $A[i][k]$ again after $\sim 2n$ (a row), $B[k][j]$ again after $\sim n^2$ (a matrix), each about a third of the time — so its miss ratio is a staircase: $2/3$ until a few values fit, $1/3$ until rows fit, near zero once a matrix fits. All of this is textbook.

What the textbook could not do is *produce that histogram for a real kernel without running it*. For half a century the distribution has been an empirical object:

how you get it	what you get
hardware counters, cache simulators	one number per (machine, input, variant) run
reuse-distance profiling (Ding et al.)	one numeric curve per profiled input; other sizes by extrapolation
polyhedral miss counting (cache-miss equations; PolyCache)	exact counts, but only with every loop bound fixed to a constant
I/O complexity (Hong-Kung)	symbolic in n and C — but only exponents: $\Omega(n^3/\sqrt{C})$, constants and thresholds gone

Note the gap: the first three rows have constants but no symbols; the last has symbols but no constants. Nobody could write down the thing you actually want — *the miss ratio of gemm as a formula in both the problem size and the cache size*. That formula is what this repository is about.

2. The histogram becomes a formula

Two ideas, from the algebraic-locality work this study builds on, close the fifty-year gap.

First touches were the blocker, and repetition dissolves it. The first access to each value has no previous use, hence no reuse distance. Numerically you just count those as misses; *algebraically* they poison everything — with infinite distances in the distribution, the working-set mathematics (Denning 1968) stops converging, which is the real reason the theory stayed numeric for decades. The resolution: analyze the kernel *as if it executes repeatedly*, so every first touch becomes a re-touch from the previous round — an **imaginary reuse** with a finite, computable distance. This is not a convenient fiction: time-stepped solvers and ML training loops *do* run their kernels repeatedly, and for a genuinely one-shot run the added reuses are identifiable and removable. With the distribution made finite and complete, the classical recursion is *provably exact* — no stochastic assumptions — and it inherits a conservation law

(the distances, weighted by frequency, must sum to exactly the data size) that will earn its keep below.

Counting becomes geometry. In an affine kernel — subscripts linear in loop counters: matmul, convolutions, stencils, attention blocks — the set of loop iterations whose reuse distance equals a given symbolic expression is a system of linear constraints, and counting integer points in such parametric sets is a solved problem with closed-form answers. A compiler can therefore emit, for each kernel, a short table: *reuse-distance formula*, *frequency formula* — polynomials in **all** loop bounds. Deriving a kernel’s table takes on the order of a minute, once, ever. (This tractability is not free in general: the underlying problem is provably NP-hard, and counting #P-hard, for arbitrary loop programs. The affine structure of real kernels is what makes the practical case fast — worth knowing when you wonder why this wasn’t done long ago.)

Here is the entire memory behavior of gemm, derived automatically, at cache-line granularity (64-byte lines, $n \times n$ matrices):

once the cache holds ...	... the miss ratio is
~6 lines	1/16
$n/4$ lines (two rows)	1/32
$n^2/8$ lines (one matrix)	$1/(16n)$
$3n^2/8$ lines (everything)	0

Four rows. That object — not a profile, not an exponent, a *formula family* — replaces the empirical column of the table above. The paper validated it against cycle-accurate cache simulation across 41 kernels ($\approx 1\%$ average miss-ratio error), so accuracy is settled; what a reviewer should ask instead is: **what can you do with a formula that you could not do with measurements?** The rest of this document is that list, computed over 22 PolyBench kernels plus a matmul-variant family, with nothing measured on hardware.

3. Formulas make “for all” statements

A measurement asserts a point. A formula asserts a region — and some of the most useful facts about kernels are region facts, unprovable by any finite profiling campaign:

- **gemm’s miss ratio is 3.1% at 32 KB — and provably the same at every cache size up to $n^2/8$ lines** (31 MB at $n = 2016$). L1, L2 and L3 of a real machine sit on one step of the staircase: for this kernel they are interchangeable, and money spent on the middle of the hierarchy buys nothing. A profiler would show you 3.1% at the sizes you tried; the formula says *there is nothing to find in between*.
- **Row-parallel matmul moves the same total data at every worker count.** Substituting the sliced bound (Section 5) yields aggregate traffic constant in p — 1.02 billion lines whether p is 1 or 1008. Not “we didn’t observe an improvement”: *independent of p* , as an identity of the formulas.
- **Tiling gemm is worth exactly nothing once $n \leq \sqrt{C/8}$ bytes** — 64 for a 32 KB cache, 362 for 1 MB, 2048 for 32 MB — and worth 36–106x above it (Section 4). “Should I tile?” finally has an honest answer, and it is not yes or no; it is a condition with the machine in it.

The cliff schedule in the last bullet generalizes: every kernel’s staircase boundaries invert into closed-form critical sizes $n(C)$, *the problem sizes at which a given machine changes regime. That is scaling behavior in the sense practitioners need it — not the exponent, but where the behavior changes and what it costs when it does**.

4. Formulas correct the accepted summaries

If the formulas only confirmed intuition, a reviewer could shrug. They do not.

Asymptotic classification gives wrong answers, not just vague ones. Our own earlier study on this suite classified kernels by the growth exponent of their data movement — the standard complexity-style summary. It placed trmm and trisolve in the same class: “no locality headroom, leave them alone.” The formulas split them: trisolve truly has nothing to gain (its miss ratio is pinned at 3.1% with *no* intermediate step until all data fits — no tiling can help), while trmm misses 50% at 32 KB against 3.1% at 1 MB — a sixteen-fold waste sitting exactly where the exponent view certified nothing was possible. Same class, opposite truths, separated only by the constants and thresholds that asymptotics discards.

No single number can rank kernels — provably, and it misranks in practice. It is tempting to compress each table into one “total data movement” score. The conservation law forbids it from working: since distance $\times$ frequency sums to the data size, any score that weights long distances is dominated by rows carrying a *vanishing* fraction of accesses — precisely the rows real caches never see. Concretely: gesummv scores worse than mvt (33.8 vs 29.4) yet misses 3.3x *less* at 32 KB; kernels with scores within 5% differ by up to 6x in actual traffic. The failure of our earlier analysis was not sloppiness; it was using a scalar where the object is a staircase.

A folklore rule becomes a theorem with a validity interval. The “ $\sqrt{2}$ rule” (Hartstein et al.): doubling the data requires $\sqrt{2}$ $\times$ the cache to hold the miss ratio. In the formulas this is exactly the statement that the active staircase boundary scales as the square root of the data size — true for ten of the 22 kernels *in a specific, computable cache interval* (for gemm: between the row step and the matrix step), false below it, false above it, and false at every size for trisolve, which has no square-root step at all. The rule of thumb survives — with its domain of validity attached, per kernel, which is what an engineering rule was always missing.

5. Formulas compose: parallelism by substitution

Because the tables are symbolic in *each loop bound separately*, they answer questions nobody analyzed them for. Give each of p cores a contiguous slice of one loop, and the per-core access stream *is* the original kernel with that bound divided by p . Parallel memory behavior = substitution into existing formulas. For matmul at $n = 2016$:

- **Slice the i-loop** (rows of the output): the re-swept matrix B appears whole in every worker’s formula, so no private cache ever gets relief — the flat-in- p identity of Section 3. A thousand cores, zero traffic reduction, known before running anything.
- **Slice the j-loop** (columns): the per-worker working set does carry the $1/p$, and the formulas predict that p private 1 MB caches begin acting as one large cache at $p = n^2 \times 8 B / 1 \text{ MB} \approx 32$. Substituting confirms: at $p = 32$, total traffic collapses 61x. The same substitution shows the collapse *cannot* happen at 32 KB — a sliced matrix still spans one 64-byte line per row, so its footprint never drops below n lines = 126 KB — and that slices narrower than the 8-value line lose spatial locality and traffic climbs again. The cache-line floor, the classic silent killer of naive parallel reasoning, falls out of the algebra unasked.

Same arithmetic, same core count; a 61x traffic gap between the two decompositions, with the threshold $p = 32$ and the 126 KB floor derived in closed form. The tiling family reads the same way — the derived variants’ tables, divided row by row, give the gain *as a function of cache size* (36x to 106x at practical sizes; tile-32 four times worse than tile-8 below its 24 KB working set; all variants exactly 1.0x once a matrix fits) — turning tile-size selection from an autotuning search into an inequality.

6. Formulas audit themselves

An empirical curve that is wrong looks exactly like an empirical curve that is right. A formula cannot hide: the frequencies must sum to the access count, and the conservation law must hit the data size — both checkable mechanically. Run over the suite, these checks *caught* six kernels (the time-stepped stencils: jacobi, heat-3d, seidel, fdtd) whose analyzer output violates conservation by up to 54%; they are excluded from every number above and reported, not plotted. The checks even localize damage: gemm’s table is short exactly one matrix worth of long-distance mass, which touches only the final staircase step and nothing quoted here. One caveat survives auditing: for triangular kernels (cholesky, lu, syrkm) the current distance approximation under-resolves the longest reuses, and claims about them are withheld until an exact mode lands. A framework that can convict its own implementation is doing something profilers cannot.

7. What this opens

The asymmetry of Section 1 is gone: for affine kernels, the memory side of performance is now a derived, closed-form object — small enough to print, exact enough to invert, symbolic enough to compose. The uses demonstrated here from one derivation per kernel:

- miss ratios for every cache size and problem size at once, replacing per-configuration simulation sweeps;
- closed-form performance cliffs $n^*(C)$ — scaling behavior with the constants in;
- transformation decisions (interchange: 4.5x flat; tiling: 36–106x) *with their validity conditions*, ending yes/no folklore;
- corrections to asymptotic classification (trmm) and a proof that no scalar locality score can be trusted;
- rules of thumb ($\sqrt{2}$) upgraded to theorems with per-kernel validity windows;
- parallel decomposition analysis by parameter substitution — provable uselessness of one split, the exact cache-merging core count of another, and line-granularity floors, none requiring new analysis;
- and mechanical self-verification that catches broken output instead of plotting it.

Arithmetic complexity has been symbolic since we learned to count operations. This line of work makes the memory side — the side that limits these kernels on real machines — symbolic too: the program is the model, a compiler extracts it in a minute, and questions that used to cost a measurement campaign become algebra.

Appendix: provenance and reproduction

All numbers derive from the AutoLALA analyzer (`dmd-cli`) under the paper’s canonical configuration — infinite repeat, scale approximation in Barvinok, 64-byte lines of eight f64 values — over the extracted PolyBench DSLs in `dsl/` plus matmul variants (naive ijk/ikj, tiles 8/16/32) written for this study. The naive-matmul run also reproduces the paper’s element-granularity Table 1 exactly, and its block-8 staircase matches the paper’s Table 6 in structure (`tables/anchors.md`). Suite evaluations use $n = 2016$ (2048 for the matmul family); caches are counted in 64-byte lines (32 KB = 512 lines). The paper’s own evaluation establishes accuracy against Cachelgrind/Dinero and hardware counters (~1% average miss-ratio error; 99.6% data-movement accuracy).

Pipeline: `run_suite.py` (analyzer runs, ~10 min) → `regimes.py` (exact symbolic extraction: piecewise quasi-polynomial parsing, exact polynomial fits with held-out verification, scale clustering; Fraction arithmetic throughout; entries gated by their region domains) → `derived.py`, `parallel_study.py`, `anchor_checks.py` → `tables/` (`machine_map.md`, `tiling.md`, `par-`

allel.md, signatures.md, dmd_inversion.md, suite_regimes.md, anchors.md, summary.json). Conservation excluded heat-3d (1.54), jacobi-1d (1.33), jacobi-2d (1.27), seidel-2d (0.93), fdtd (0.96), imperfect (0.97); convolution is analyzed as a 9x9-filter variant (image and filter are genuinely different scales and cannot both be bound to n). Raw analyzer JSON (data/, 29 MB) is gitignored and regenerable.

```
python3 run_suite.py && python3 regimes.py && python3 derived.py
python3 parallel_study.py && python3 anchor_checks.py
pandoc REPORT.md -o REPORT.pdf --pdf-engine=xelatex \
  -V mainfont="DejaVu Serif" -V monofont="DejaVu Sans Mono" \
  -V geometry:margin=1in
```